lecture 5

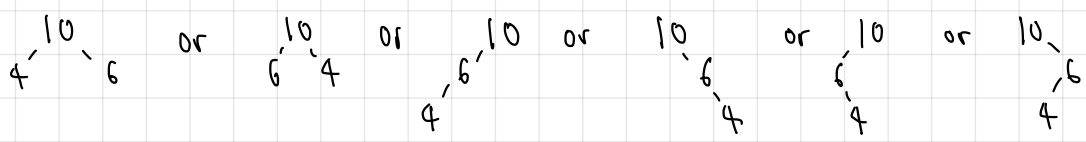
Heap sort
- which team at the current time has the most goals
- even as the num of elements and the values change

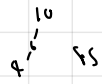
dynamic max problem
- the problem with maintaining a sorted list in a dynamic problem is that updates, insertions, deletions, reordering each take O(n)
- find max takes O(1)

- use a heap (max by default)

Heap (max)
- binary tree where every node is greater than its children
  ↳ or equal

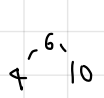
given 6, 4, 10
heaps:



note, a one sided heap  is just like an array
however, this is not good because the run time is the same as a sorted array. we want a balanced

---

heaps are not binary search trees,
↑ in BST, left is smaller and right is bigger
are binary trees (each node has 2 children)



they are balancing issues to permit a faster list structure

Completeness for binary tree

1. all layers but the last are filled
2. the nodes in the last layer are all to the left



each layer doubles the number of nodes



$2^0 = 1$
$2^0 + 2^1 = 3$
$2^0 + 2^1 + 2^2 = 7$

Sum of geometric series: $2^k - 1$ nodes in k layers

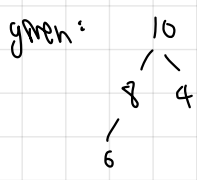
$n = 2^k - 1$
$n+1 = 2^k$
$\log_2(n+1) = \log_2 2^k$
$k = \log_2(n+1)$ k layers given n nodes

Insertion
1. put at last layer's left most position
2. compare element with its parent and swap if needed (if node is greater than parent)
   - swimming

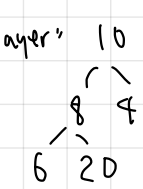
needs to put at location to maintain completeness

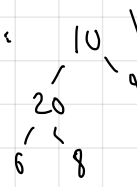
allow nodes to swim to the correct location

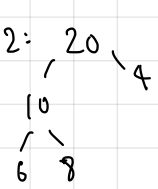
Def swim(x):
  while x.value > parent(x).value
    swap x ↔ parent(x)

given:  , insert 20

---

last layer: 

swim 1: 

swim 2: 

Def insert(value):
  x = new node
  x.value = value
  insert x on last layer of heap
  swim(x)

worst case: $O(\log(n+1))$ swaps
$\approx O(\log(n))$

Deletion

needs to ensure order is maintained

delete at root ensures sinking. delete elsewhere might result in swimming

idea

1. fix shape
2. then worry about order | swap last layer's leftmost, maintains structure
depending on the value of it, sink or swap
sink: do the opposite of swimming
- go downward. swap with larger child

deleting at the root only sinks, because smaller elements TF moving down

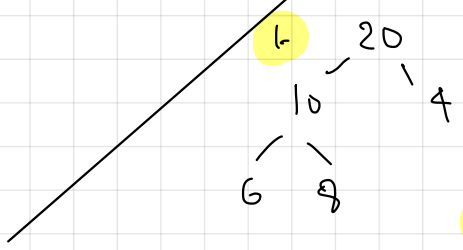
deleting elements other than heap regains swimming or sinking

Sinking
- compare node with children's value

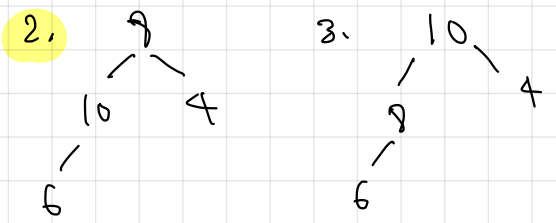
```
def sink(x):
    while x.value < larger_child(x).value:
        swap x <-> larger_child(x)
```

```
def delete(heap, x)
    l = rightmost leaf in last layer of heap
    if x is l:  # base case
        remove x from heap
    else:
        old_value = x.value
        swap x.value <-> l.value
        remove l from heap
        if x.value > old_value:
            swim(x)
        else:
            sink(x)
```

delete 20



1. put 8 at root
2. fix order, if root is smaller than children swap with larger child. sink



Heap can not find the k-largest item

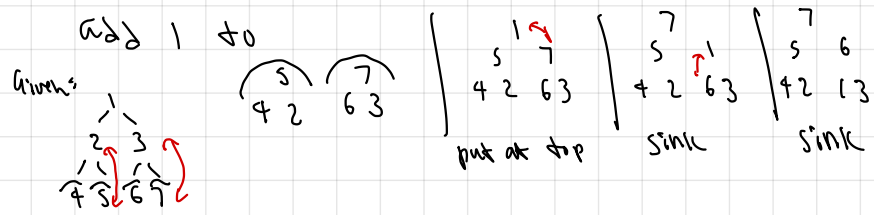
Heap: Convert complete binary tree to heap

```
def heapify(tree):
    for level in bottom level-1, ..., top level:
        for node in level:
            sink(node)
```

Value:
O(nlog(n))
start from bottom and sink each node

---

put at top and sink it needed



---

heap sort
1. find max
2. swap it to the back
3. repeat

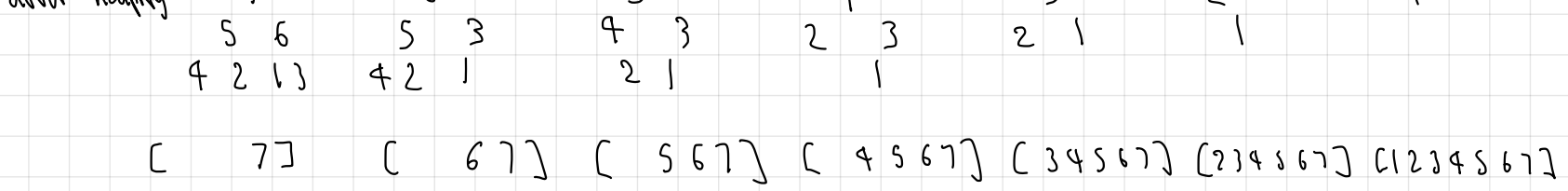
like selection sort
1. find min
2. swap it to the front
3. repeat

```
Def heap_sort(A):
    heap = complete binary tree of elements of A   X
    heapify(heap)
    for i in 0,1,...,n-1:
        delete root's heap element and place at A[n-1-i]
        # maintains heap property
```

O(n)
O(n)
n times
log(n)
O(nlog(n))

Total: O(nlog(n))

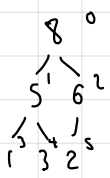
after heapify:



Why?
heap sort reuses the array to store the heap

Given

array:
[8 5 6 1 3 2]
index: 0 1 2 3 4 5



node 5 is index 1:
left child: 2i+1
right child: 2i+2

parent node: $\lfloor \frac{i-1}{2} \rfloor$

last leaf: N-1

store a heap in an array
derive parent and children indices